

2nd Assessment Report

Anh + Anh_id

1. Introduction

The smart home system manages devices for comfort, convenience, and energy savings. It allows remote/auto control and real - time monitoring. Python's OOP is applied. The `Device` base class encapsulates common features and methods. `Light`, `Thermostat`, `Camera` inherit from it, adding unique traits. The `DeviceController` manages devices via a dictionary. The singleton `SmartHomeHub` coordinates tasks, status display, and energy calcs for an integrated system.

This project helps learners master practical OOP and design patterns. In a smart home system build, they apply OOP (e.g., inheritance for reuse) and the singleton for the hub to optimize architecture.

First, devices (lights, thermostats, cameras) form the system base. The device class has common traits, and subclasses add unique ones, showing object - oriented traits.

Second, the `DeviceController` manages devices with a dictionary, enabling addition, removal, and control, integrating data structures and object - oriented programming.

Third, the `SmartHomeHub` uses the singleton pattern. It schedules tasks, displays status, calculates energy, and coordinates components, helping learners see design patterns' role in system stability.

2. Device Class Design

In the smart home system, the Device class is the base for Light, Thermostat, and Camera. It offers a common template and basic functions. Sub - classes can reuse its attributes and methods via inheritance, reducing code duplication and improving maintainability and extensibility.

Device has attributes: `__device_id` for unique identification, `__name` for user convenience, `__status` (initially 'off', changed by `turn_on/off`), and `__energy_usage` (starts at 0, set by `set_energy_usage`).

In the `'Device'` class, attributes are made private with `'__'`. Getter methods (e.g., `'get_id()'`) let external code safely access values, and setter methods (e.g., `'set_status()'`) enable secure modification, with room for validation. This encapsulation protects data, enhances maintainability, and offers flexibility for extra features.

`turn_on()` method

Function: Sets the device's status to "on". Defined in the base Device class, subclasses like Light, Thermostat, and Camera can use it to turn on the device.

`turn_off()` method

Function: Sets the device's status to "off". Defined in the base Device class for subclasses to turn off the device.

```
# Control methods
#turn on the device
def turn_on(self):
    self.__status = "on"

#turn off the divece
def turn_off(self):
    self.__status = "off"
```

`get_energy_usage()` method

- Function: Retrieves the device's energy usage value. Defined in the base Device class, returning the `__energy_usage` attribute. Subclasses can override it for specific calculations (not overridden in current code).

```
def get_energy_usage(self):
    return self.__energy_usage
```

In this smart home system code, Light, Thermostat, and Camera classes inherit from the Device class. They call Device's constructor via `super().__init__(device_id, name)` to inherit its private attributes accessed by getters/setters. Although no method overriding exists currently, subclasses could override methods like `turn_on()` for specific functions. Each subclass also adds unique attributes, like `brightness` for Light, showing polymorphism.

```
# Subclasses for devices
#we will define the light class, thermostat class and camera class by inheriting from the 'Device' class.
class Light(Device):
    def __init__(self, device_id, name, brightness=100):
        super().__init__(device_id, name) #initialization
        self.brightness = brightness
```

3. Device Controller Class Design

In the smart home system code, dictionaries play a crucial role in device management. For instance, in the DeviceController class, the devices dictionary uses `device_id` as keys and device objects as values. This allows for easy addition of devices via `add_device`, removal via `remove_device`, listing by iterating values, and command execution by fetching devices using `device_id`. Dictionaries are efficient for lookup as they are based on hash tables. Python calculates key hashes for storage and retrieval, resulting in an average $O(1)$ time - complexity. This ensures quick device lookups even with a large number of devices, as seen in the `execute_command` method of DeviceController.

1. `add_device(device)`

Function: Adds a device to the DeviceController's device dictionary. It gets the device's ID and uses it as the key to store the device in the dictionary.

Command Example:

```
controller = DeviceController()
light = Light('L1', 'Living Room Light')
controller.add_device(light)
```

Expected Response: The light device is added to the controller's devices dictionary with key 'L1'.

2.

remove_device(device_id)

- **Function:** Removes a device from the dictionary if the given device ID exists.

- **Command Example:**

```
controller = DeviceController()
light = Light('L1', 'Living Room Light')
controller.add_device(light)
controller.remove_device('L1')
```

Expected Response: If 'L1' was added, it is removed from the controller's devices dictionary.

○

3. **list_devices()**

- **Function:** Lists all devices in the DeviceController. If the dictionary is empty, it shows a prompt. Otherwise, it prints device info.

- **Command Example:**

```
controller = DeviceController()
light = Light('L1', 'Living Room Light')
controller.add_device(light)
controller.list_devices()
```

4. **Expected Response:** If devices are present, it prints their info. If empty, it prints "Oops! No devices added..."

execute_command(device_id, command)

- **Function:** Executes "on" or "off" commands on a device if it exists. Prints an error for invalid commands.

```
controller = DeviceController()
light = Light('L1', 'Living Room Light')
controller.add_device(light)
controller.execute_command('L1', 'on')
```

- **Expected Response:** If the device exists and the command is valid, its status changes. Otherwise, it prints an error.

4. Smart Home Hub Class Design

In a smart home system, a single `SmartHomeHub` instance is vital. It manages resources, avoiding chaos from multiple instances, ensures data consistency, and simplifies architecture with a unified interface for developers, enhancing maintainability and extensibility. By overriding `\_\_new\_\_`, it checks for an existing instance and returns it, enabling centralized command and resource management.

SmartHomeHub uses the singleton pattern. A private constructor and `\_\_instance = None` stop new external instances. The overridden `\_\_new\_\_` makes one instance and always returns it. The `DeviceController` unifies device management (e.g., `display\_status` uses `list\_devices`). `schedule\_task` manages scheduling to avoid conflicts, keeping the system running smoothly.

`schedule_task(device_id, command, time):`

The `schedule_task()` method can arrange device tasks. It takes in three parameters: `device_id`, `command`, and `time`. It will print out information about the device executing the specified command at the specified time.

```
#print the working information of a specific device.
def schedule_task(self, device_id, command, time):
    print(f"scheduled task: device {device_id} will obey {command},at {time}")

scheduled task: device C1 will obey on,at 2025-03-24 16:34:27
```

The `display_status()` method can show the status of all devices in the system. It first prints a prompt message and then calls the `list_devices()` method of the `DeviceController`. If the device dictionary

is not empty, it will iterate through and print the information of each device.

```
def display_status(self):  
    print("status of all devices:")  
    self.controller.list_devices()
```

```
status of all devices:  
Oops! No devices added to the controller yet. Add some first.
```

total_energy_usage():

The total_energy_usage() method calculates the total energy consumption of all devices. It creates a device iterator and achieves this calculation through the internal recursive function.

```
def total_energy_usage(self):  
    #create a iterator which will allow us to iterate through each device one by one.  
    devices_iterator = iter(self.controller.devices.values())
```

The nested recursive_calculate function recursively fetches devices from the iterator, sums their energy usage, and terminates when the iterator is exhausted (caught by StopIteration).

```
def recursive_calculate(devices_iterator):  
    try:  
        #recursively call function get total energy usage of each device and sum them up  
        device = next(devices_iterator)  
        return device.get_energy_usage() + recursive_calculate(devices_iterator)  
    except StopIteration: # if all devices have been processed  
        return 0
```

The method returns the result of the recursive calculation, which sums the energy usage of all devices.

```
return recursive_calculate(devices_iterator)
```

In the smart home system, the SmartHomeHub communicates with devices via its internal DeviceController instance, created during initialization for a unique reference. For example, display_status() calls DeviceController's list_devices() to show device status, and execute_command() uses it to send commands for on/off control.

First, SmartHomeHub's `schedule_task()` takes device ID, command and time to schedule tasks, boosting energy efficiency. Second, it centrally manages devices, gets status and energy data via recursive `total_energy_usage()`, enabling auto - adjustment for optimal energy use. Third, its unified interface simplifies system use and development for status query, command execution and task scheduling.

5. Testing and Demonstration

The code uses Python's built - in `'unittest'` to test the smart home system. It provides tools for unit tests. A `'TestSmartHomeSystem'` class inheriting from `'unittest.TestCase'` has multiple `'test_'` - prefixed methods auto - executed by the framework. For instance, `'test_device_operations'` tests device on/off and uses `'assertEqual'` for result checks.

`unittest` edge - case testing checks software at special boundaries. In smart - home tests, it includes re - operating active devices, handling non - existent ones, and unsupported commands. Design test cases for these to find hidden defects and boost software stability in complex scenarios.

But, I think the test code here lacks edge - case testing.

Automated testing is key for software reliability. In complex projects like smart home systems, manual testing lags and errs. It runs cases fast, finds defects, detects code - change issues, stabilizes functions, improves consistency, spots issues early, cuts costs, and boosts quality.

Test Implementation are as below:

1. test_device_operations

Verifies Light device's on - off operations. Checks initial off state, on state after turning on, and off state after turning off.

```
def test_device_operations(self):
    self.assertEqual(self.light.get_status(), 'off')
    self.light.turn_on()
    self.assertEqual(self.light.get_status(), 'on') #status assertion
    self.light.turn_off()
    self.assertEqual(self.light.get_status(), 'off') #status assertion
```

2. test_controller_add_remove_device

Tests DeviceController's add/remove device functions. Checks if device ID exists after addition and is absent after removal.

```
def test_controller_add_remove_device(self):
    self.controller.add_device(self.light)
    self.assertIn('L1', self.controller.devices)
    self.controller.remove_device('L1')
    self.assertNotIn('L1', self.controller.devices)
```

3. test_controller_execute_command

Validates DeviceController's command execution. Checks device state after sending on and off commands.

```
def test_controller_execute_command(self): #Verify the result of command execution
    self.controller.add_device(self.light)
    self.controller.execute_command('L1', 'on')
    self.assertEqual(self.controller.devices['L1'].get_status(), 'on')
    self.controller.execute_command('L1', 'off')
    self.assertEqual(self.controller.devices['L1'].get_status(), 'off')
```

4. test_hub_schedule_task_and_energy_usage

Tests SmartHomeHub's task scheduling and energy consumption calculation. Schedules a task, calculates energy, and compares with manual sum.

```
def test_hub_schedule_task_and_energy_usage(self):
    task_time = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    self.hub.schedule_task('L1', 'on', task_time)
    total_energy = self.hub.total_energy_usage()
    self.assertEqual(total_energy, sum(device.get_energy_usage() for device in self.hub.controller.devices.values()))
```

5. test_device_inheritance_and_methods

Verifies device class inheritance and methods. Checks object - class instance relationships and operation results.

```
def test_device_inheritance_and_methods(self):
    self.assertIsInstance(self.light, Light) #Check inheritance relationship
    self.assertIsInstance(self.thermostat, Thermostat)
    self.assertIsInstance(self.camera, Camera)
    self.light.turn_on()
    self.assertEqual(self.light.get_status(), 'on')
    self.thermostat.turn_off()
    self.assertEqual(self.thermostat.get_status(), 'off')
```

From the test results, 5 test cases were run in 0.001 seconds with an “OK” result. This shows the smart home system functions normally under current conditions, with no issues found.

```
PS C:\Users\hyx> & D:/python的文件/python.exe "c:/Users/hyx/Desktop/Assessment 2/test.py"
....scheduled task: device L1 will obey on,at 2025-03-24 18:03:29
.
-----
Ran 5 tests in 0.001s

OK
PS C:\Users\hyx> █
```

6. Conclusion

This project boosted my Python OOP skills. I designed device classes with encapsulation, inheritance and a standard `\_\_str\_\_` method. Used dictionaries for O(1) management and a Singleton hub. Solved energy sum via recursion, overcame iteration issues. Testing confirmed its scalability and trade - offs.

I initially faced challenges implementing recursive energy calculation with iterator state and termination issues. By reviewing course materials and researching on StackOverflow, I debugged step - by - step. Fixing errors deepened my recursion understanding, showing its pros and cons, which will guide future algorithm choices.

The program is designed to make home services more intelligent. Currently, the activity attributes of devices are rather monotonous. To enhance user experience, a user - defined scenario function can be introduced. For instance, lights can automatically turn on at 8 a.m. to wake the user, and the thermostat can start when it's over 30°C for comfort.